

AI Resume Screening & Ranking

Evaluator Answer Key — results, rationale, and full walkthrough

Generated 2026-09-23 17:05 • run over 50 candidate resumes • 41 eligible / 9 rejected

Summary of this run

Metric	Value	Metric	Value
Resumes ingested	50	Eligible	41
Parsed OK	50	Rejected	9
Parse failed	0	Duplicates skipped	0
LLM used	True	LLM scored (survivors)	41
Skipped by hard gate	9	LLM rate-limit waits	0
GitHub enriched	6	GitHub no profile	25
GitHub tagged (rate-limited)	10	GitHub tagged (error)	0
Duration (s)	419.37	Rank 1 score	99.5

What this system is

A small, production-minded backend that ingests a folder of resumes, applies a **hard, deterministic eligibility filter first** (Python AND AI/agentic evidence), scores the survivors on a 100-point rubric, enriches them with public GitHub activity, uses an LLM only to judge **project quality with cited evidence**, and returns a ranked shortlist where **every point is explained**. It runs with zero API keys (deterministic mode) and scales up gracefully when a Gemini key and GitHub token are provided.

Headline outcomes of this run

- **50 resumes parsed**, 0 duplicates, 0 unreadable — the batch completed with no crashes.
- **41 eligible, 9 rejected**, each rejection carrying an explicit reason.
- **9 candidates were eliminated before any network call**, so the LLM and GitHub were only used on candidates that could actually rank.
- LLM project scoring applied to **41** survivors; 6 GitHub profiles enriched, 25 had no profile.

How the submission maps to the evaluation rubric

Rubric area	Weight	How this submission addresses it
Resilience (handling server issues)	25%	content-hash dedupe; dual PDF extractor; a deterministic positive-only gate that rejects non-Python / non-AI projects
Ranking (project quality)	25%	source weighting; thin-wrapper (5-15) and tutorial (3) penalties; framework-name-in-skills capped at 10/40; full per-project quality logic
Added complexity (Pydantic), evidence-backed scores, adapter pattern, env-only keys, per-resume failure isolation, rate-limit wait-and-resume, response	15%	
Code quality & architecture	15%	config separated from logic, Pydantic contracts throughout, 66 tests, typed schemas, clean boundaries between ingest/extract/
Activity (0-5) maintained/relevant	10%	pos (0-5), capped at 10; never fatal; failures tagged with a neutral placeholder; in-run circuit breaker on rate limit
Reliability & tests covering matching	5%	precision, gate ordering, scoring, penalties, adapter pacing, ingestion edge cases, determinism; failure-mode ma
README Detailing	5%	README (quickstart), .env.example, requirements.txt, CLI + optional FastAPI, generated results.j

1. End-to-end pipeline

Stage	What happens	Why it is here
1. Ingest	Discover files, de-duplicate by content hash, parse PDFs, DOCX, TXT	Pre-processed per file so a malformed resume cannot abort the batch
2. Deterministic extract	Regex + vocabulary: name, email, GitHub, skills, rough projects, available degree (the LLM validated base)	On projects, available degree (the LLM validated base) instead of a blank
3. HARD GATE (Python AND AI)	Eligibility decided from raw-text evidence only	Cheapest filter, run first; eliminated candidates cost no API calls
4. GitHub enrichment	Only for survivors with a username; never eliminates	Additional positive signal, not an eligibility requirement
5. LLM scoring	Optional: scores each AI project's depth + rationale + Sentiment (judge)	Where it adds most value; off the critical path; degraded
6. Score + rank	Combine categories, apply penalties, clamp 0-100, rank	One deterministic final number per candidate with a full rationale trail

Why this order matters

- **Gate before the LLM** — a candidate with no Python or no AI evidence is eliminated before any network call; the model never sees candidates that cannot rank.
- **Frozen eligibility** — because eligibility is computed before the LLM, the model can neither grant nor revoke it. The hard filter is provably outside the model.
- **GitHub before the LLM** — independent enrichment that overlaps with LLM pacing.

2. Eligibility — the hard gate

A candidate is **eligible only if both** hold, decided by deterministic code on raw-text evidence: **(a) Python evidence** — Python or a Python-only framework appears as a real token; and **(b) AI/agentive evidence** — a core LLM/agentive signal, or a genuine AI project (agentive or general ML/CV/NLP) inside a project/experience section.

Positive-only, never negative

The gate asks only “is Python present?” and “is AI present?”. It never rejects *because* JavaScript, Java, React or Next.js is present — those only contribute supporting score and appear in `matched_skills`. Rejection reasons name only the unmet requirement.

Resume profile	Outcome
JavaScript / React only	Rejected — missing Python AND AI
Java / Spring Boot / React only	Rejected — missing Python AND AI
JavaScript + React + Python + AI	Eligible (JS is irrelevant to the gate)
Python + FastAPI, no AI project	Rejected — missing AI
AI project written in JS/TS, no Python	Rejected — Python is a hard requirement

AI evidence tiers

Tier	Examples	Counts as AI?
1 — LangChain, LlamaGuard, LangChain, LlamaGuard	Google ADK, RAG, embeddings, vector search, tool calling, multi-agent, evaluation, fine-tuning, anywhere	Yes — anywhere
2 — AI ecosystem	LLM, foundation model, GenAI, Hugging Face, Bedrock, SageMaker, ASR/TTS	Yes — anywhere
3 — general ML/DL	machine learning, deep learning, CNN/RNN, XGBoost, scikit-learn, TensorFlow, PyTorch, NLP, vision, data project / experience section	Yes — anywhere

Matching precision (false positives are the enemy)

- **Java ≠ JavaScript, SQL ≠ PostgreSQL, “CV” is never matched** (it means curriculum vitae) — enforced by word boundaries.
- “embedded systems” ≠ “embedding”; “data model” ≠ AI model; “travel agent” ≠ AI agent.
- Glued and hyphenated forms tolerated: “MachineLearning”, “multi-agent”, “multi agent”.

3. Scoring — the 100-point rubric

Category	Weight	How it is earned
AI / Agentic / RAG project depth	40	best project depth (70%) + breadth of solid projects (15%) + rubric signals (15%)
Python & backend engineering	30	weighted terms × evidence-source multiplier
Cloud / deployment / full-stack	15	Docker/GCP/AWS/K8s/CI-CD; React/Next as supporting
GitHub activity	10	0-5 recent activity + 0-5 maintained/relevant repositories
Engineering depth signals	5	testing, architecture, caching, queues, observability...
Project-quality penalties	-5...-15	thin LLM/API wrappers; tutorial/undetailed projects

The core idea: evidence-source weighting

Every matched term records **where** it was found, and the same term earns different credit — directly implementing the spec's requirement to reward evidence in projects/internships over keyword-only skill lists.

Where the term appears	Multiplier
Project description	1.0
Internship / work experience	0.9
Summary / preamble	0.5
Skills list only	0.4

Why these penalties

- **Thin wrapper (-5...-15):** an “AI project” that is really a single LLM/API call with no retrieval, orchestration, tools, evaluation, state or backend workflow.
- **Tutorial / undetailed (-3):** named like a tutorial, or described too briefly to show implementation detail or ownership.
- **Framework name in a skills list only** is capped at $0.25 \times 40 = 10$, so naming alone can never reach the top.
- A classical ML project (e.g. a CNN) is **not** treated as a thin wrapper.

4. How and why the LLM is used

- **Scope:** score each AI project's depth (0-10), return the rubric signals, a one-sentence rationale, and a short cited-evidence snippet. It does not decide eligibility or extract fields.
- **Structured output:** a Pydantic response schema is enforced and validated locally; invalid output is discarded and the deterministic value is used.
- **Evidence required:** a score without a rationale and citation is not trusted.
- **Small prompts:** only the PROJECTS + EXPERIENCE context is sent, not the whole resume.
- **Adapter:** one interface with a Gemini implementation and a null fallback; provider is swappable in one class; keys come only from the environment.
- **Rate limits:** calls are paced under the free-tier per-minute quota; on a quota error the adapter waits for the reset and resumes the same request rather than silently degrading.
- **Zero-key mode:** without a key the system still produces a full ranking via a transparent signal-based quality estimate.

5. GitHub enrichment

- Public REST API only; an optional env token raises the limit from 60 to 5000 req/hr.
- Signals: recency and volume of public events (falling back to repo push dates), maintained repositories, and Python/AI-relevant repositories.
- Scoring: 0-5 activity + 0-5 repositories, hard cap 10. Never affects eligibility.

- **Tagging:** a rate-limited or errored profile is *unknown*, not inactive, so it is tagged and given a deterministic neutral placeholder (5/10). A genuinely absent profile still scores 0. No random values — they would break reproducibility.

6. Results of this run

Of **50** resumes, **41** were eligible and **9** were rejected. The gate removed **9** candidate(s) from every network call; **41** survivor(s) received LLM project scoring. GitHub was enriched for **6**; **25** had no profile and **10** were tagged (rate-limited/error).

Ranked shortlist (eligible candidates)

#	Candidate	Total	AI	PY	Cloud	GH	Eng	Pen
1	Yash Maini	99.5	40	30	15	10	5	0
2	PRAJWAL A S	88.2	39	30	6	8	5	0
3	V Sree Raghu Vardhan	88.0	38	30	15	0	5	0
4	Hari Shanker Sharma	86.5	36	30	15	0	5	0
5	ARUNIMA SAHA	85.6	36	30	15	0	5	0
6	ABHINAV MISHRA	85.0	30	27	13	10	5	0
7	Gourab Das	85.0	30	30	15	5	5	0
8	Vivek Chimnani	83.3	29	29	15	5	5	0
9	Prathamesh Patil	82.2	35	30	15	0	5	-3
10	Sumaiya Sultana Shaik	81.7	32	30	15	0	5	0
11	Thejeshwaar Paasila	81.5	27	30	15	5	5	0
12	Kiran Naregal	78.8	36	24	13	5	4	-3
13	Nevil Jasmin Mehta	78.6	29	30	15	0	5	0
14	Aditi Kala	78.0	33	29	15	5	2	-6
15	Arjun Kumar	75.2	27	30	8	10	1	0
16	RAJ THUMAR	75.0	29	30	11	0	5	0
17	Vaibhav Wakde	74.1	32	30	7	0	5	0
18	SHUBHANSHU NAVADIYA	70.2	36	30	4	0	2	-3
19	PAVANI A	69.9	21	30	13	5	1	0
20	Jyandeep Baishya	69.7	35	25	5	0	4	0
21	ANNISHA S	69.1	26	30	7	5	1	0
22	Agam Jain	68.7	25	30	10	0	4	0
23	Vivek Shekhawat	68.0	29	24	5	5	5	0
24	PRIYANSHU KUMAR	65.8	37	23	3	0	2	0
25	HARSHA A	64.3	26	30	7	0	1	0
26	M SAI KUSHITH	63.5	32	29	1	0	1	0
27	KRISHAN KANT	61.0	25	23	8	0	5	0
28	P.V. Jayavardhana Sai	60.1	33	17	8	0	2	0
29	Yuvaraj R	59.9	18	28	10	0	4	0
30	NIDHI KM	53.5	20	19	5	5	4	0
31	Aswin Raj R	49.4	25	19	4	0	1	0
32	NISARGA H. S.	46.6	36	9	2	0	0	0
33	VISHWAS BADIGER	46.0	19	24	2	0	1	0
34	DORNIPADU KARTHIKA CHAITRIKA	45.5	25	15	0	5	0	0

#	Candidate	Total	AI	PY	Cloud	GH	Eng	Pen
35	Saif Ali Nagori	44.6	16	19	3	5	1	0
36	JAYA NANDI	43.7	17	20	6	0	1	0
37	VIKRAM MADHYASTA	42.7	15	8	14	0	5	0
38	Parshuram Kumar	41.9	2	22	13	0	5	0
39	Atul Kumar	40.7	20	27	8	0	0	-15
40	PRIYA R	37.4	23	10	1	1	2	0
41	MANOJ KUMAR K R	35.1	15	18	1	0	1	0

Why the top candidates ranked here (evidence trail)

#1 Yash Maini — 99.5/100

Project: CopyGuard: A serverless GenAI code detection platform built on AWS Lambda and Bedrock. It provides low-latency detection for burst traffic with full infrastructure-as-code implementation.

GitHub: last public activity ~3d ago; 2 event(s) in 90d; relevant repos: face-verification-resnet50, pg-reliability-agent

- CopyGuard: depth 5.0/10 — The project demonstrates solid production-grade infrastructure and backend engineering for a GenAI service, though it lacks complex agentic features.
- Candida Hyphae Segmentation: depth 6.0/10 — The project shows significant depth in model architecture design and data pipeline optimization for medical imaging.
- NetoAI Solutions - Voice Agent: depth 8.0/10 — This project involves complex real-time orchestration and low-latency engineering for a production-grade voice agent.
- NetoAI Solutions - Sales Copilot: depth 10.0/10 — This is a highly sophisticated agentic system featuring multi-agent orchestration, complex tool use, RAG, and rigorous testing.

#2 PRAJWAL A S — 88.2/100

Project: AI Voice Appointment Assistant: A real-time, multilingual voice AI agent for healthcare appointment management. It utilizes WebRTC for low-latency interaction and integrates with a PostgreSQL database via FastAPI to ensure grounded tool-calling for appointment

GitHub: last public activity ~47d ago; relevant repos: voice-ai-backend, python-low-level-design, medical-ai-assistant

- AI Voice Appointment Assistant: depth 7.0/10 — The project demonstrates strong backend integration and tool-calling architecture to ensure data consistency in a real-time voice application.
- AI-Powered HPC Monitoring Assistant: depth 8.0/10 — This project features sophisticated multi-agent orchestration and complex data handling for high-performance computing environments.
- CSIR-4PI HPC Infrastructure & AI Systems: depth 10.0/10 — This is a highly mature, production-deployed system featuring rigorous evaluation, complex data pipelines, and multi-stage agentic workflows.
- best-depth 28.0 + breadth 6.0 (3 solid project(s)) + signals 5.2

#3 V Sree Raghu Vardhan — 88.0/100

Project: AI-driven enterprise HRMS workflows: A complex system of multi-step AI agents integrated with enterprise APIs, semantic retrieval, and document processing to automate HR tasks. It features structured output validation, Redis-backed state management, and async

GitHub: No GitHub profile on resume

- AI-driven enterprise HRMS workflows: depth 10.0/10 — This is a highly sophisticated production-grade system featuring multi-step orchestration, state management, rigorous evaluation pipelines, and complex backend integration.
- Business Requirements Document generation platform: depth 5.0/10 — The project demonstrates solid backend engineering and integration of LLMs for a specific business use case, though it lacks the advanced agentic complexity of the
- best-depth 28.0 + breadth 4.0 (2 solid project(s)) + signals 6.0

#4 Hari Shanker Sharma — 86.5/100

Project: Sasta Signals: A distributed price monitoring system that tracks products across 7 major Indian e-commerce platforms using automated web scraping. It features real-time price drop detection with notifications and a robust backend deployed via Docker.

GitHub: No GitHub profile on resume

- 500Agents: depth 9.0/10 — Demonstrates high-level agentic orchestration using LangGraph, multi-provider routing, and session persistence at scale.
- Jug.ai: depth 8.0/10 — Includes complex features like content training pipelines, real-time intent detection, and AI-to-human handoff logic.
- Support.cc: depth 7.0/10 — A functional business-logic-heavy application integrating RAG-like knowledge management with automated triage workflows.
- TheAIBull.com: depth 9.0/10 — High-stakes system integrating real-time financial data, complex backtesting engines, and secure multi-broker API orchestration.

#5 ARUNIMA SAHA — 85.6/100

Project: Multi-Agent Travel Planning System: An agentic travel planning and booking system that uses LangGraph to orchestrate specialized nodes for flight/hotel search and booking. It features human-in-the-loop workflows, persistent state management via PostgreSQL, and *GitHub*: No GitHub profile on resume

- Multi-Agent Travel Planning System: depth 9.0/10 — This is a sophisticated multi-agent system featuring complex orchestration, human-in-the-loop workflows, and robust state management.
- Graph RAG Conversational Assistant: depth 8.0/10 — The project demonstrates advanced RAG techniques using knowledge graphs and a multi-stage pipeline for query generation and result formatting.
- AI-Based Text-to-SQL Chatbot: depth 8.0/10 — This project shows strong engineering maturity through its decoupled serverless architecture, security practices, and robust logging/validation layers.
- best-depth 25.2 + breadth 6.0 (3 solid project(s)) + signals 4.5

Rejected candidates (9) and the exact reason

Candidate	Reason(s)	Matched skills (evidence)
AABHAR GUPTA	No evidence of Python stack; No AI/agentic project evidence	PostgreSQL, MongoDB, Redis, REST API, Node.js, Express, JavaScript, GCP
ANJALI PATIL	No evidence of Python stack; No AI/agentic project evidence	PostgreSQL, MySQL, Redis, SQL, REST API, Java, Spring Boot, JavaScript
Gayathri Palraj	No AI/agentic project evidence	Python, MySQL, MongoDB, SQL, REST API, Node.js, Express, Java
Harshit Kumar Mishra	No evidence of Python stack; No AI/agentic project evidence	PostgreSQL, MongoDB, Redis, SQL, GraphQL, WebSockets, Node.js, Express
Kartikay Sinha	No evidence of Python stack; No AI/agentic project evidence	MySQL, MongoDB, SQL, REST API, Node.js, Express, C++, TypeScript
MD AZIZUL ISLAM	No evidence of Python stack	OpenAI API, RAG, Embeddings, pgvector, LLM, PostgreSQL, MongoDB, Redis
Mukul Pal	No evidence of Python stack	OpenAI API, LLM, PostgreSQL, MongoDB, Redis, REST API, WebSockets, Node.js
SHIVAM RAJ	No evidence of Python stack; No AI/agentic project evidence	REST API, Node.js, Express, TypeScript, JavaScript, Azure, Docker, CI/CD
Shreya Chauhan	No AI/agentic project evidence	Python, MySQL, MongoDB, Redis, SQL, REST API, Kafka, Node.js

7. Validation & edge cases handled

Behaviour	Evidence
Malformed / unreadable PDF	Recorded as failed; batch continues (test + synthetic fixture)
Empty / image-only file	Recorded as empty; rejected with a reason
Duplicate file (same bytes)	De-duplicated by content hash (test)
Two-column / glued-word PDF	Both pdfplumber and pypdf run; higher-quality text wins
Missing name / email / GitHub	Safe fallbacks (email local part, filename)
LLM error or invalid JSON	Per-resume warning; deterministic score used (test)
LLM rate limit	Waits for reset and resumes; verified live (10 waits, 33/33 survivors scored)
GitHub rate limit / private	Tagged + neutral placeholder; batch continues (test)
Eliminated candidate	LLM and GitHub never called (spy-client test)
Determinism	Same input -> identical scores (test)

Bugs found on the real 50-resume set and fixed

- Two-column PDFs were interleaved by one extractor — fixed by running both extractors and picking the higher-quality text.
- Dropped spaces inside some PDFs (“andbuildingproduction”) — fixed by the same dual-extractor choice.
- “Machine Learning” / “semantic chunking” weren't recognised, wrongly rejecting real ML projects — vocabulary extended (incl. glued forms).
- A section heading (“TECHNICAL SKILLS”) was accepted as a candidate name — heading/lowercase rejection added.
- The LLM briefly influenced eligibility via its projects — reverted so eligibility uses only raw-text evidence (regression test added).
- A thin chatbot escaped the wrapper penalty because “LangChain” was counted as orchestration — corrected.

8. Key design decisions & trade-offs

- **Deterministic-first, LLM-optional.** Everything except project-quality judgement is deterministic, so the system is reproducible, testable, and works with zero keys.
- **Gate-first.** Filtering before enrichment saves API budget and keeps the model out of eligibility; the trade-off is that a vocabulary miss is silent, which is mitigated by tiering, glued-form handling, and auditable reasons.
- **Evidence-source weighting** makes skill credit depend on context, matching the spec's intent rather than rewarding keyword stuffing.
- **Tagged GitHub failures with a neutral placeholder** avoid mistaking a rate limit for inactivity while keeping scores deterministic.

Known limitations

- Rejected candidates' project names are noisy — they never reach the LLM by design.
- No OCR: scanned/image-only resumes are not recovered.
- Section detection is heuristic; unusual headings may land in a catch-all bucket.
- GitHub without a token hits 60 req/hr and gets tagged.
- LLM score calibration is prompt-driven (no labelled ground-truth set).

If I had more time

- Cut LLM latency (disable thinking budget, cap output tokens).
- Higher throughput (second key/model, or batch several resumes per scoring call).
- Incremental runs via a per-candidate result cache keyed by file hash.
- One-request GraphQL GitHub; a labelled calibration set; OCR + column-aware parsing.

9. How to run / reproduce

Deterministic only (no keys):

```
python main.py --input ./resumes --output ./output/results.json --no-llm
```

Full run (LLM + GitHub; requires GEMINI_API_KEY / GITHUB_TOKEN in .env):

```
python main.py --input ./resumes --output ./output/results.json --csv ./output/results.csv --pdf  
./output/results.pdf --process-pdf ./output/process_report.pdf
```

Environment: `pip install -r requirements.txt`; tests: `pytest -q` (66 tests).

Optional API: `uvicorn src.api:app --reload` (POST /screen, GET /results).

Appendix — lower-ranked eligible candidates (rank 21+)

#	Candidate	Total	AI	PY	Cloud	GH	Pen
21	ANNISHA S	69.1	26	30	7	5	0
22	Agam Jain	68.7	25	30	10	0	0
23	Vivek Shekhawat	68.0	29	24	5	5	0
24	PRIYANSHU KUMAR	65.8	37	23	3	0	0
25	HARSHA A	64.3	26	30	7	0	0
26	M SAI KUSHITH	63.5	32	29	1	0	0
27	KRISHAN KANT	61.0	25	23	8	0	0
28	P.V. Jayavardhana Sai	60.1	33	17	8	0	0
29	Yuvaraj R	59.9	18	28	10	0	0
30	NIDHI KM	53.5	20	19	5	5	0
31	Aswin Raj R	49.4	25	19	4	0	0
32	NISARGA H. S.	46.6	36	9	2	0	0
33	VISHWAS BADIGER	46.0	19	24	2	0	0
34	DORNIPADU KARTHIKA CHAITRIKA	45.5	25	15	0	5	0
35	Saif Ali Nagori	44.6	16	19	3	5	0
36	JAYA NANDI	43.7	17	20	6	0	0
37	VIKRAM MADHYASTA	42.7	15	8	14	0	0
38	Parshuram Kumar	41.9	2	22	13	0	0
39	Atul Kumar	40.7	20	27	8	0	-15
40	PRIYA R	37.4	23	10	1	1	0
41	MANOJ KUMAR K R	35.1	15	18	1	0	0

Full machine-readable results are in [output/results.json](#); a flat summary in [output/results.csv](#); a results-only PDF in [output/results.pdf](#); a process/design PDF in [output/process_report.pdf](#).